

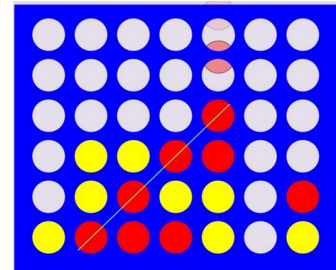
TP : Création d'un Agent pour le Jeu Connect Four

Objectif :

Dans ce TP, vous allez implémenter un agent pour jouer au jeu Connect Four. Le but est de développer un agent qui peut jouer de manière simple, puis améliorer son comportement en utilisant des heuristiques et, finalement, le Deep Reinforcement Learning (RL).

1. Présentation du Jeu Connect Four

Connect Four (aussi appelé **Four in a Row**) est un jeu de société classique qui se joue sur une grille de 6 lignes et 7 colonnes. Le but du jeu est d'être le premier à aligner quatre de ses jetons (disques) de manière consécutive, soit horizontalement, verticalement ou en diagonale.



Règles du jeu :

- Deux joueurs jouent à tour de rôle. Un joueur utilise des jetons rouges, l'autre utilise des jetons jaunes.
- À chaque tour, un joueur choisit une colonne pour y déposer un jeton. Le jeton tombe dans la première case vide de la colonne.
- Le jeu se termine dès qu'un joueur aligne quatre jetons de manière consécutive, soit :
 - Horizontalement : 4 jetons alignés sur la même ligne.
 - Verticalement : 4 jetons alignés sur la même colonne.
 - Diagonalement : 4 jetons alignés en diagonale.
- Si la grille est remplie sans qu'aucun joueur n'ait gagné, la partie se termine par un match nul.

Stratégies possibles :

- Les joueurs doivent anticiper les mouvements de leur adversaire pour éviter qu'il ne gagne et pour créer des opportunités d'aligner leurs propres jetons.
 - Un bon agent devra prendre en compte les coups à jouer pour gagner, mais aussi pour bloquer son adversaire.
-

2. Objectif du TP

Dans ce TP, vous allez implémenter un agent pour jouer au jeu Connect Four. L'objectif est de créer un agent qui :

- Peut jouer contre un autre joueur.
 - Utilise des stratégies de plus en plus sophistiquées, allant d'un agent aléatoire à un agent basé sur des heuristiques et, enfin, un agent utilisant le Deep Reinforcement Learning.
-

3. Déroulement du TP

Partie 1 : Création de l'Environnement de Jeu

Dans cette première partie, vous allez créer l'environnement de jeu Connect Four. Cela inclut la représentation de la grille, la gestion des règles de placement des jetons, et la vérification des conditions de victoire.

Instructions :

1. Implémentez une grille de 6 lignes et 7 colonnes. La grille est initialement vide.
 2. Créez une fonction permettant de placer un jeton dans une colonne. Le jeton doit tomber dans la première case vide de la colonne.
 3. Créez une fonction pour vérifier si un joueur a gagné après chaque coup. Un joueur gagne s'il aligne quatre jetons horizontalement, verticalement ou en diagonale.
 4. Implémentez une fonction pour afficher la grille après chaque coup.
-

Partie 2 : Création d'un Agent Simple

Dans cette partie, vous allez créer un agent qui choisit ses coups de manière aléatoire.

Instructions :

1. Créez une fonction qui permet à l'agent de jouer un coup au hasard dans une colonne valide.
 2. Après chaque coup, l'agent doit jouer son tour et la grille doit être mise à jour.
 3. L'agent joue contre un joueur humain. Le joueur humain choisit une colonne et l'agent joue son coup au hasard.
-

Partie 3 : Création d'un Agent Sophistiqué

Dans cette partie, vous allez améliorer l'agent en lui permettant de choisir un coup qui pourrait le faire gagner si possible.

Instructions :

1. Modifiez l'agent pour qu'il choisisse un coup gagnant si possible (si un coup permet de compléter une ligne de quatre jetons).
 2. Si aucun coup gagnant n'est disponible, l'agent choisit un coup au hasard parmi les colonnes disponibles.
 3. L'agent joue contre un joueur humain, qui choisit une colonne pour placer son jeton.
-

Partie 4 : Implémentation d'une Heuristique

Dans cette partie, vous allez implémenter une heuristique pour améliorer l'agent. L'agent doit évaluer chaque coup possible en fonction de l'heuristique et choisir celui qui maximise son score. Une **heuristique** est une fonction qui attribue un score à une configuration donnée du plateau. Ce score reflète la probabilité qu'une configuration mène à une victoire ou à une défaite.

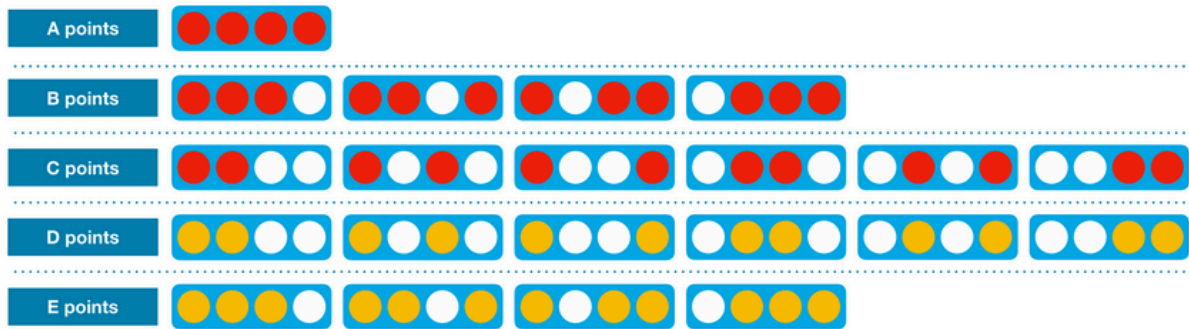
Fonctionnement de l'heuristique

L'heuristique que vous allez implémenter repose sur l'évaluation des "fenêtres" (groupes de 4 cases adjacentes) sur le plateau. Voici les éléments pris en compte :

1. Nombre de fenêtres contenant 4 jetons alignés pour le joueur :
Une telle configuration correspond à une victoire immédiate.
Score = A * num_fours
2. Nombre de fenêtres contenant 3 jetons alignés pour le joueur :
Ces configurations sont très prometteuses, car elles peuvent mener à une victoire en un coup.
Score = B * num_threes
3. Nombre de fenêtres contenant 2 jetons alignés pour le joueur :
Ces configurations représentent des opportunités de développement.
Score = C * num_twos
4. Nombre de fenêtres contenant 2 jetons alignés pour l'adversaire :
Ces configurations peuvent être dangereuses si elles permettent à l'adversaire de progresser.
Score = D * num_twos_opp
5. Nombre de fenêtres contenant 3 jetons alignés pour l'adversaire :
Ces configurations sont critiques, car elles peuvent conduire à une défaite imminente.
Score = E * num_threes_opp

Le score total de l'heuristique est donné par la formule suivante :

$$\text{score} = A * \text{num_fours} + B * \text{num_threes} + C * \text{num_twos} + D * \text{num_twos_opp} + E * \text{num_threes_opp}$$



Comprendre la logique de l'heuristique :

- Pourquoi est-il important de maximiser les scores pour certaines configurations et de minimiser ceux pour d'autres ?

Définir les coefficients A,B,C,D,E :

- Choisissez des valeurs pour ces coefficients afin que l'agent joue efficacement.
- Par exemple, une victoire immédiate (4 jetons alignés) doit avoir un score beaucoup plus élevé qu'une opportunité de développement (2 jetons alignés).

Implémenter la fonction d'heuristique :

- Utilisez la structure suivante dans votre code :

```
def get_heuristic(grid, mark, config):
    num_twos = count_windows(grid, 2, mark, config)
    num_threes = count_windows(grid, 3, mark, config)
    num_fours = count_windows(grid, 4, mark, config)
    num_twos_opp = count_windows(grid, 2, mark % 2 + 1, config)
    num_threes_opp = count_windows(grid, 3, mark % 2 + 1, config)
    score = A * num_fours + B * num_threes + C * num_twos + D * num_twos_opp
    return score
```

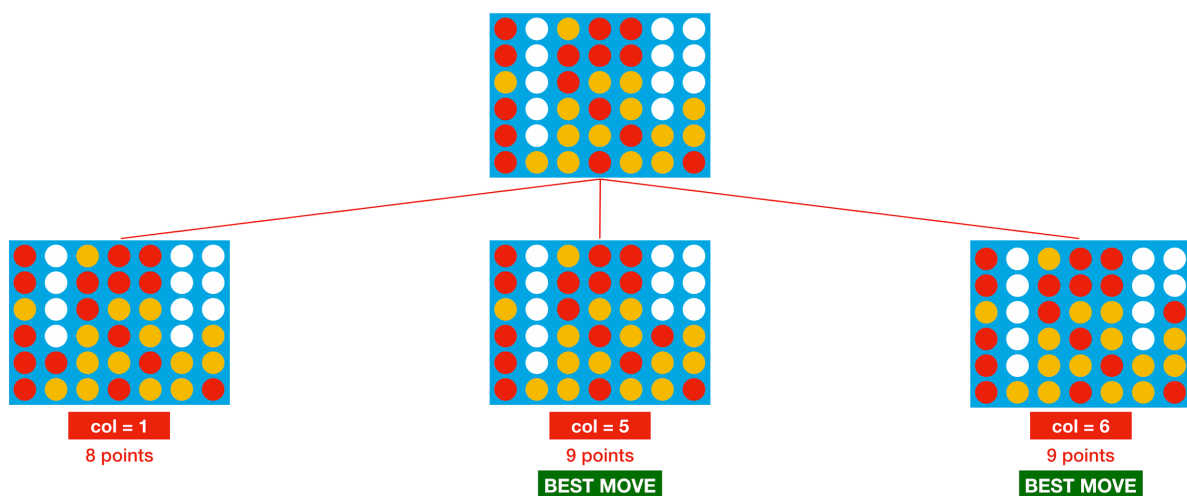
Critères de réussite

- L'agent doit gagner au moins la moitié des parties jouées contre l'agent de base.
- Les coefficients C et D doivent être non nuls (les configurations à 2 jetons doivent être prises en compte).

Partie 5 : Implémentation d'une Deuxième Heuristique

Dans le TP précédent, vous avez appris à développer un agent utilisant une recherche avec **lookahead à un pas**. Bien que cet agent fonctionne correctement, il reste des lacunes. Par exemple, il est incapable de prévoir les conséquences de ses actions sur plusieurs tours, ce qui peut conduire à des erreurs stratégiques.

Prenons l'exemple suivant : si l'agent rouge doit choisir entre deux colonnes, il pourrait opter pour une colonne qui permet à l'adversaire de gagner au tour suivant, simplement parce qu'il ne regarde qu'un seul coup en avant. Pour résoudre ce problème, nous allons introduire l'algorithme **Minimax**, qui permet à l'agent de planifier ses actions en regardant plusieurs tours dans le futur.



1. Rappel : Lookahead à un pas

Reprenez l'agent développé précédemment qui utilise une recherche à un pas pour choisir ses coups. Cet agent :

- Simule chaque coup possible.
- Évalue immédiatement l'impact de chaque coup à l'aide d'une fonction heuristique.
- Choisit le coup avec le meilleur score.

2. Introduction à l'Algorithme Minimax

L'algorithme **Minimax** repose sur les principes suivants :

- L'agent cherche à **maximiser** son score.
- Il suppose que l'adversaire joue de manière optimale pour **minimiser** le score de l'agent.
- L'arbre des décisions est exploré jusqu'à une certaine profondeur (par exemple, 3 niveaux).
- À chaque niveau :

- Les nœuds où l'agent joue sont des **nœuds MAX**.
- Les nœuds où l'adversaire joue sont des **nœuds MIN**.

3. Implémentation de Minimax

- Implémentez une fonction récursive pour explorer l'arbre des décisions.
- Utilisez une profondeur fixe (par exemple, 3) pour limiter la recherche.
- Intégrez une fonction heuristique pour évaluer les états du jeu (par exemple, en comptant les alignements possibles de 4 jetons pour chaque joueur).

Partie 6 : Amélioration de l'Agent avec Deep Reinforcement Learning

Dans cette dernière partie, vous allez utiliser le Deep Reinforcement Learning pour entraîner un agent à jouer au Connect Four.

Instructions :

1. Implémentez un agent basé sur le Deep Reinforcement Learning en utilisant une méthode comme Q-learning ou un réseau de neurones pour prédire les actions.
2. Entraînez l'agent en lui permettant de jouer contre lui-même ou contre un autre agent.
3. Évaluez la performance de l'agent en le faisant jouer contre un joueur humain.